

Multi-Source Learning Content Ingestion & Structured Output Generation

Comprehensive System Architecture, Functional Feature Mapping & Technical Reference

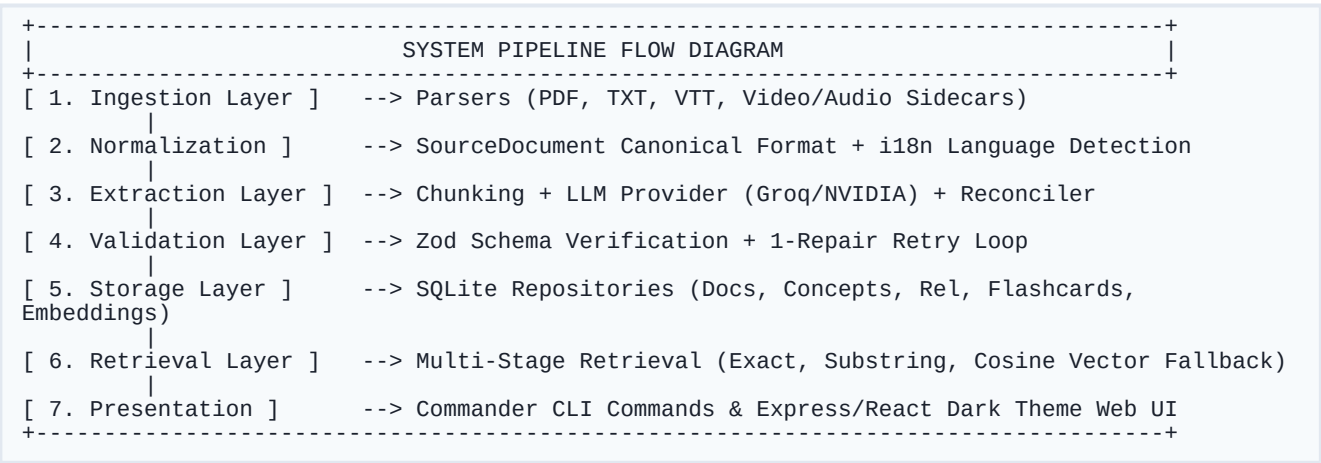
1. Executive Overview & System Objectives

The Multi-Source Learning Content Ingestion System is an enterprise-grade platform built to solve the challenge of transforming unstructured educational materials into structured, interactive learning artifacts. By accepting diverse input formats—including PDFs, plain-text transcripts, VTT subtitle tracks, and video/audio transcript sidecars—the platform extracts key concepts, topological prerequisite relationships, concise document summaries, flashcard study decks, and vector embeddings for semantic search.

Designed around a strict unidirectional layer architecture, the system guarantees zero circular dependencies, provider-agnostic LLM extraction (supporting Groq, NVIDIA NIM, and Anthropic), schema validation with automatic repair retries, cross-document concept deduplication, and a dual-interface presentation layer comprising a Commander CLI and a dark-mode React web application.

2. Complete System Architecture

The architecture comprises seven isolated, sequential layers. Each layer communicates exclusively through defined TypeScript interfaces, preventing cross-layer coupling and enabling independent extensibility.



Layer Responsibilities & Design Patterns

- **Ingestion Layer** (`src/ingestion/`): Implements the Plugin Pattern via `registry.ts` and `Parser` interface. Translates raw files into structured raw text and metadata.
- **Normalization Layer** (`src/normalization/`): Converts parsed outputs into the canonical `SourceDocument` schema. Evaluates document language script distribution (`src/ingestion/language.ts`).
- **Extraction Layer** (`src/extraction/`): Uses token/character chunking, LLM provider abstractions (`GroqProvider`, `NVIDIAProvider`), and a second-pass multi-chunk reconciliation prompt (`mergeChunks.ts`).
- **Validation/Structuring Layer** (`src/validation/`): Validates raw LLM JSON outputs against Zod schemas. Triggers a single repair retry on malformed JSON before raising typed errors.
- **Storage Layer** (`src/storage/`): Repository pattern over SQLite (`better-sqlite3`). Implements cross-document deduplication via lowercased `canonical_name` matching and `concept_documents` junction table.
- **Retrieval Layer** (`src/retrieval/`): Provides multi-stage retrieval: exact match -> substring match ->

semantic nearest-neighbor vector search fallback via `concept\_embeddings`.

- **Presentation Layer** (`src/cli/` & `web/`): Standalone Commander CLI for offline demo safety, paired with Express REST API and a React Web UI featuring SVG concept graph pan/zoom and topological learning paths.

Running the Project

Order of operations for local development:

```
# 1. Install & configure environment
npm install && cp .env.example .env

# 2. Initialize database schema
npm run db:init

# 3. Run CLI ingestion & learning path commands
npm run cli -- ingest seed-data/pdfs/neural_networks.pdf
npm run cli -- learning-path "Artificial Intelligence"

# 4. Launch web server & UI
npm run build && npx vite build --prefix web/client && npm run server
```

Scaling & Future Roadmap

- **Database Migration:** SQLite repositories can be swapped for PostgreSQL / Supabase with zero changes to business logic layers.
- **Async Job Queues:** Large document processing can be offloaded to Redis-backed BullMQ job queues with WebSocket progress notifications.
- **Vector Search Scaling:** Local 128-dim TF-IDF n-gram embeddings can scale seamlessly to PGVector or Qdrant for millions of concepts.
- **Parallel Extraction:** Chunk extractions execute concurrently via worker pools, merged by second-pass LLM reconciliation.
- **Retrieval Caching:** Redis LRU caching layer for sub-millisecond retrieval responses on popular topic graphs.

Internationalization (i18n)

Document Language Detection automatically flags non-Latin content. Future multi-lingual support extends extraction prompts to output English canonical concept names alongside localized names (`canonical\_name\_en`), while UI components reference JSON translation catalogs (`locales/en.json`).

3. Functional Feature Mapping (MVP & Completed Stretch Features)

MVP Core Features (100% Completed)

- **M1: PDF Parsing:** Extracts clean text from PDF documents using pdf-parse with fallback streams (FR1.1).
- **M2: Transcript Text Parsing:** Ingests text and markdown transcripts into clean string representations (FR1.2).
- **M3: Source Document Normalization:** Maps parsed files into canonical SourceDocument objects (FR1.3).
- **M4: LLM Concept & Relationship Extraction:** Extracts concepts, prerequisite/related-to/part-of relationships, and summaries (FR2.1).
- **M5: Schema Validation & Repair Retry:** Zod schema validation with automatic single-repair re-prompt on malformed JSON (FR2.2, FR2.3).
- **M6: Chunking & Merge Logic:** Token-threshold text chunking with concept deduplication (FR2.4).
- **M7: Flashcard Generation:** Generates concept Q&A flashcards exportable in JSON and CSV formats (FR3.1).

- **M8: Summary Generation:** Produces concise summaries per source document (FR3.2).
- **M9: Concept Graph Data:** Transforms concepts and relationships into node/edge graph data structures (FR3.3).
- **M10: SQLite Storage Layer:** Persists all entities via thin repository patterns with foreign keys enabled (FR4.1).
- **M11: Topic-Based Retrieval:** Queries stored artifacts by concept name with exact and substring matching (FR4.2).
- **M12: Standalone CLI Interface:** Commander CLI commands: ingest, list-topics, export (FR5.1-FR5.3).
- **M13: React Web UI:** Dark-theme UI featuring file dropzone, summary view, flashcard decks, and SVG graph (FR6.1-FR6.3).
- **M14: Setup Reproducibility:** Clean setup verified with seed data and comprehensive documentation.

Completed Stretch Features (100% Completed)

- **S1: Edge-Case Hardening:** Non-English script detection and second-pass LLM multi-chunk concept reconciliation.
- **S2: Cross-Document Deduplication:** Canonical concept name deduplication across distinct documents via junction table.
- **S3: SVG Graph Visualization Polish:** Interactive SVG canvas with pan/zoom controls, node click-to-expand panel, and edge legends.
- **S4: Semantic Vector Retrieval:** 128-dimensional n-gram TF-IDF embeddings stored in SQLite with cosine similarity search fallback.
- **S5: Video & Audio File Ingestion:** `videoParser.ts` for MP4, MP3, WAV, and VTT/SRT subtitle track ingestion.
- **S6: Comprehensive Automated Test Suite:** 13 unit and integration test modules executed via `npm run test`.
- **S7: Topological Learning Path Generation:** Computes step-by-step ordered learning paths using Kahn's topological sort algorithm.

4. API & CLI Usage Guide

The platform provides complete CLI and REST API parity:

```
# CLI Commands:
npm run cli -- ingest <filePath>
npm run cli -- list-topics
npm run cli -- export <topic> -f <json|csv>
npm run cli -- learning-path <topic> -f <text|json>

# REST API Endpoints:
POST /api/ingest           (Multipart file upload -> JSON pipeline response)
GET  /api/topics           (Returns list of distinct stored topic names)
GET  /api/topics/:topic    (Returns concepts, flashcards, summary, graph & learning path)
```

5. Database Schema Summary

Relational SQLite schema managed via `src/storage/db.ts`:

- **documents:** id (PK), filename, source_type, raw_text, metadata (JSON), ingested_at
- **concepts:** id (PK), document_id (FK), name, description, canonical_name
- **concept_documents:** concept_id (FK), document_id (FK) - Composite PK for cross-document dedupe
- **relationships:** id (PK), from_concept_id (FK), to_concept_id (FK), type
- **flashcards:** id (PK), concept_id (FK), question, answer
- **summaries:** id (PK), document_id (FK), summary_text
- **concept_embeddings:** concept_id (PK, FK), embedding (JSON vector array)

6. Known Limitations

- **Non-English Support:** System explicitly detects and rejects non-Latin scripts; native multi-language translation is deferred.
- **Binary Media Transcriptions:** Video/audio parsing requires VTT/SRT subtitle sidecars unless local Whisper binary is installed.

